verkodo

Truth code – mathematics and law written formally, checked by machine

Whitepaper 0.3

1. Summary

verkodo writes knowledge down formally instead of as prose – mathematics first, then German and, later, European law. A concept can have several definitions, joined by bridges that prove them equivalent; a lecture, a guide or a commentary is a path through them; Lean 4 checks the proofs. Text, PDF and web pages are generated from it, and every term in them explains itself – the links come from the formalisation, nobody sets them by hand. It is for people who teach, learn, write or apply mathematics and law.

The aim is one source from which a textbook, a lecture, a guide to a statute and the code that applies the statute all follow – and a machine that says which parts of it hold. A first prototype does this for a small piece of analysis and a simplified tax rule (section 6).

Why now. Three things have changed. Formal mathematics has become large enough to build on: Lean's library Mathlib holds more than 290,000 theorems and 140,000 definitions [1]. Machines now write mathematics, too – an AI system has found formal proofs at the level of the International Mathematical Olympiad [2] – so what counts is less who wrote a statement than whether it is checked. And governments ask for rules that machines can read: the OECD describes *Rules as Code*, a machine-consumable version of rules next to their text [3], and in Germany every federal regulatory project goes through a check of whether it can be carried out digitally [4]. What is missing is the link between the formal object and the text people read – verkodo is that link.

Reading guide: sections 1–3, 6 and 9 are for everyone; sections 4 and 5 are for readers who want the technical picture.

2. The problem

Mathematics is written twice – and checked once, if at all. A textbook and a formal library describe the same mathematics without knowing of each other. The textbook is read by people but checked by nobody; the library is checked by a machine but read by few. Books define the same concept in different ways – convergence of a sequence with ε and N in a first course, with filters in a library – and leave it to the reader to see that the definitions agree. A lecture that changes the definition has to be rewritten by hand, and whether its proofs still hold is a matter of trust.

Law is written as text – and run as code nobody can read. Taxes, benefits and fees are computed by software that someone translated from the statute. The translation is not part of the law, it is rarely published, and errors show up only when they hit someone: when the French family benefits were written down in a programming language for the law, the authors found a bug in the official implementation [5]. People who apply a statute – administrations, advisers, courts, citizens – cannot see whether the program says what the law says.

DRAFT

Explanations are links set by hand. Learning platforms and wikis explain a term by linking to an article about it [6, 7]. The links are made by editors, they break when the article changes, and they do not tell which definition a text actually uses.

3. The idea

verkodo describes knowledge once, formally, and derives everything else from it.

- **Concepts with variants and bridges.** A concept can have several definitions (variants). A bridge proves two variants equivalent. A theorem says which variant it is written for; over a bridge it holds for the other variant, too – checked, not assumed.
- **Paths.** A lecture, a guide or a commentary is a path through the knowledge: an order of steps, each choosing a variant. verkodo checks that every step uses only what the path has introduced before.
- **Generated text.** Text, PDF and web pages come from the formalisation, in any language a term has a label for. Every term is linked to where it comes from – on the web by hovering, on paper by a reference to its step or to an appendix.
- **Checked by machine.** Mathematical statements and proofs are checked by Lean 4. Rules of law are typed, can be run on cases, and their properties are proved. What is not proved yet is marked open – in the source and in every page made from it.
- **Two logic profiles on one knowledge layer.** Mathematics and law share concepts, labels, sources, remarks, paths and stable ids, but not their logic: mathematics uses Lean 4 and Mathlib, law a small typed rule language with defaults and exceptions. One logic for everything would serve neither.

4. How it works

A verkodo source is a set of `.vk` files. A file declares a module; every declaration gets a stable id. Shared declarations carry labels per language, sources and remarks; a path lists its steps.

Mathematics. A concept has variants whose formulas are Lean 4; a bridge carries the proof that they agree:

```
concept converges {
  params lean "(a : ℕ → ℝ) (L : ℝ)"
  variant eps_n  { lean "∀ ε > 0, ∃ N : ℕ, ∀ n ≥ N, |a n - L| < ε" }
  variant filter { lean "Filter.Tendsto a Filter.atTop (nhds L)" }
  bridge eps_n <-> filter { proof lean "…" }
}

theorem limit_unique {
  params lean "(a : ℕ → ℝ) (L M : ℝ)"
  using converges.filter
  statement lean "converges a L → converges a M → L = M"
  proof lean "fun h₁ h₂ => tendsto_nhds_unique h₁ h₂"
}
```

`verkodo prove` turns each module into a Lean file: every variant becomes a definition, every bridge a theorem that the two definitions are equal. For a path that uses the ε-N variant, `limit_unique` – written for filters – is carried over along the bridge and checked again by Lean. Lean's verdict (checked, open, rejected) comes back onto the line of the `.vk` file. verkodo also asks Lean which constant each name in a formula stands for, so Mathlib names such as `tendsto_nhds_unique` explain themselves with their type and documentation, like verkodo's own terms.

Law. A norm has inputs, constants and rules; a rule has a default and exceptions, tried in order:

```
norm small_business {
  valid from 2025-01-01
  input turnover_prev : money
  constant limit_prev : money = 25_000 EUR
```

DRAFT

```
  rule within_limits : bool = turnover_prev <= limit_prev
  rule exempt : bool {
    default resident and within_limits
    unless waived then false
  }
}
```

(Simplified; inputs left out. The limits and the waiver were checked against the official text of § 19 UStG on 2026-11-06.)

Amounts carry a currency and are exact; rounding is part of the language, as the law prescribes it (§ 32a EStG rounds twice). A provision may have versions, each valid for a time, and verkodo checks that they follow one another without gap. Undetermined legal terms ("expected turnover") are judgements – inputs someone has to assess, with their criteria –, and every rule shows which judgements its result depends on. Obligations, permissions and prohibitions are rules that say whether they apply. `verkodo prove` turns a norm into Lean functions of its inputs, so properties of the law are ordinary theorems – "a waiver always excludes the exemption", "at the limit the business is still exempt". If a rule changes so that such a property no longer holds, Lean rejects the proof, and the error lands on the theorem in the `.vk` file.

Writing and output. verkodo can be written with braces and quotes or by indentation alone, and `verkodo fmt` turns one into the other. A module names the languages it is written in; other languages come from translation files. `verkodo render` writes HTML in which every term has a hover card; `verkodo pdf` sets a path as a PDF through Typst, where every term links to its step or to an appendix that explains it. Both mark what is open. In an editor, `verkodo lsp` shows the same cards on hover, and in a proof the goals Lean sees at each tactic; the web editor checks in the browser and evaluates norms step by step for given inputs.

5. Related work

verkodo takes ideas from several projects and differs from each in one point.

Project	What it does	How verkodo differs
AMBOSS [6]	medical knowledge platform; terms in articles linked to their definitions	links come from the formalisation, not from editors; contents are checked
nLab [7]	collaborative wiki for research mathematics, strong on category theory	formal and checked; several definitions of a concept joined by proved bridges
MMT / OMDoc [8]	module system for formal knowledge: theories as nodes of a graph, views mapping one into another	verkodo takes the concepts – theory graphs, views as bridges – and binds them to one prover (Lean) and to generated text
Lean Blueprint [9]	LaTeX document of a formalisation project with a dependency graph and the Lean status of each node	the text is generated from the formal source instead of being written next to it; paths instead of one document
Verso [10]	Lean's authoring tool: documents are Lean files, Lean code shown with hovers	verkodo is written for readers who do not write Lean, and for law as well as mathematics
Catala [5]	programming language for the law, written next to the statute's text	verkodo shares law and mathematics on one knowledge layer and proves properties of rules in Lean
Lean 4 and Mathlib [1, 11]	prover and its library of formal mathematics	verkodo builds on them; it adds variants, bridges, paths and text

6. Where it stands

verkodo is at its beginning. Version 0.2.0 is on crates.io (`cargo install verkodo`, Rust 1.99 or later; Lean and Typst are installed separately), in Rust, as nine crates – from the syntax to an SDK for editors and IDEs:

DRAFT

- `verkodo check` parses and resolves `.vk` files, checks types, paths, versions and translations;
- `verkodo prove` exports to Lean 4 (Mathlib), carries theorems over chains of bridges and maps Lean's verdicts, names and goals back onto the source;
- `verkodo render` and `verkodo pdf` write HTML with hover cards and PDF through Typst; `verkodo fmt` formats;
- `verkodo lsp` is the language server for editors; `verkodo serve` the web editor, with Lean on the server when switched on.

Three examples serve as proof of concept: the beginning of an analysis course (sequences, convergence with ε-N, with filters and in metric spaces, paths through the same material, a French translation), § 19 UStG in its versions of 2020 and 2025 (small businesses, the waiver and its revocation, properties of the reform proved in Lean) and the income tax scale of § 32a EStG with its roundings. On 2026-11-06 they held 61 declarations; Lean checked 62 parts, 2 are open (marked as such), none was rejected.

7. Licences and governance

verkodo is free software under AGPL-3.0-or-later, published by falkOSS – everything included, no telemetry. Texts such as this whitepaper are under CC BY 4.0; the artwork of the brand under CC BY-ND 4.0. Contributions are signed off under the Developer Certificate of Origin.

8. The name

```
vero   truth
kodo   code
       → verkodo
```

verkodo joins two Esperanto words: *vero*, truth, and *kodo*, code – a system of signs, and the text of a program [12, 13]. Put together they make a truth code: knowledge written down so that a machine can check that it holds. Said ver-KO-do, stressed on the second-to-last syllable like every Esperanto word.

9. Roadmap

1. **Libraries of knowledge** – `.vk` files outside the tool, checked by CI like Mathlib: first a core of German tax law; projects that depend on such libraries.
2. **Law in Lean** – norms that use other norms, cases that span a change of the law, who bears an obligation.
3. **Editors and output** – an extension for VS Code over the language server, LaTeX next to HTML, Typst and Markdown.
4. **First content** – a lecture and a guide to a statute written in verkodo from start to end, with people who teach and apply them.
5. **Exchange** with the communities behind MMT, Lean and Catala.

Sources

1. Lean community, Mathlib statistics. https://leanprover-community.github.io/mathlib_stats.html, retrieved 2026-11-06 (140,068 definitions, 292,546 theorems).
2. T. Hubert et al., "Olympiad-level formal mathematical reasoning with reinforcement learning", *Nature*, 2025. https://doi.org/10.1038/s41586-025-09833-y
3. OECD Observatory of Public Sector Innovation, *Cracking the Code: Rulemaking for humans and machines*, 2020. https://oecd-opsi.org/publications/cracking-the-code/
4. Digitalcheck – digitaltaugliche Regelungen, DigitalService GmbH des Bundes. https://digitalcheck.bund.de/, retrieved 2026-11-06.
5. D. Merigoux, N. Chataing, J. Protzenko, "Catala: A Programming Language for the Law", *Proceedings of the ACM on Programming Languages* 5 (ICFP), 2021. https://doi.org/10.1145/3473582

DRAFT

6. AMBOSS, "About AMBOSS". https://support.amboss.com/hc/en-us/articles/360044236212-About-AMBOSS, retrieved 2026-11-06.
7. nLab, "About". https://ncatlab.org/nlab/show/About, retrieved 2026-11-06.
8. F. Rabe, M. Kohlhase, "A Scalable Module System", *Information and Computation* 230, 2013.
9. P. Massot, leanblueprint. https://github.com/PatrickMassot/leanblueprint, retrieved 2026-11-06.
10. Verso, Lean's authoring tool. https://github.com/leanprover/verso, retrieved 2026-11-06.
11. L. de Moura, S. Ullrich, "The Lean 4 Theorem Prover and Programming Language", *CADE-28*, 2021.
12. Reta Vortaro, article *ver/a* (vero, vera). https://www.reta-vortaro.de/revo/art/ver.html, retrieved 2026-11-06.
13. Reta Vortaro, article *kod/o* (kodo). https://www.reta-vortaro.de/revo/art/kod.html, retrieved 2026-11-06.

DRAFT